Ai Assisted coding

Ht.No:2303A52350

Assignment:16.3

Batch:45

Task 1 – Hospital Management Database

Task:Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

• List all appointments for a specific doctor.

• Retrieve patient history by patient ID.

• Count total patients treated by each doctor.

Prompt:

Generate SQL code for a Hospital Management System.Create tables for Doctors, Patients, and Appointments with proper constraints like primary keys, foreign keys, and valid data checks. Insert sample data into the tables. Write SQL queries using joins to:
1. List appointments for a specific doctor
2. Retrieve patient history
3. Count total patients treated by each doctor
Provide complete working SQL code.

Code:

```
DROP TABLE IF EXISTS appointments;

DROP TABLE IF EXISTS patients;

DROP TABLE IF EXISTS doctors

CREATE TABLE doctors (

    doctor_id INTEGER PRIMARY KEY,

    name TEXT NOT NULL,

    specialty TEXT NOT NULL    );
```

```sql
CREATE TABLE patients (
    patient_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER CHECK(age > 0));
CREATE TABLE appointments (
    appointment_id INTEGER PRIMARY KEY,
    patient_id INTEGER,
    doctor_id INTEGER,
    appointment_date DATE NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id));
-- 2. INSERT SAMPLE DATA
INSERT INTO doctors VALUES (1, 'Dr. Rao', 'Cardiology');
INSERT INTO doctors VALUES (2, 'Dr. Sharma', 'Neurology');
INSERT INTO patients VALUES (1, 'Anil', 30);
INSERT INTO patients VALUES (2, 'Priya', 25);
INSERT INTO appointments VALUES (1, 1, 1, '2024-04-01');
INSERT INTO appointments VALUES (2, 2, 1, '2024-04-02');
INSERT INTO appointments VALUES (3, 1, 2, '2024-04-03');
-- 3. QUERIES
-- 1. List all appointments for a specific doctor
SELECT
    a.appointment_date,
    p.name AS patient_name
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;
-- 2. Retrieve patient history by patient ID
SELECT
    p.name AS patient_name,
    d.name AS doctor_name,
    d.specialty,
```

```
    a.appointment_date
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
WHERE p.patient_id = 1;
-- 3. Count total patients treated by each doctor
SELECT
    d.name AS doctor_name,
    COUNT(DISTINCT a.patient_id) AS total_patients
FROM doctors d
LEFT JOIN appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.name;
```

Output:

SQL ▼ 1 / 1 1 - 2 of 2

appointment_date	patient_name
2024-04-01	Anil
2024-04-02	Priya

SQL ▼ 1 / 1 1 - 2 of 2

patient_name	doctor_name	specialty	appointment_date
Anil	Dr. Rao	Cardiology	2024-04-01
Anil	Dr. Sharma	Neurology	2024-04-03

SQL ▼ 1 / 1 1 - 2 of 2

Show SQL

doctor_name	total_patients
Dr. Rao	2
Dr. Sharma	1

Explanation:

This prompt generates a Hospital Management System database with tables for doctors, patients, and appointments using proper constraints.It also creates SQL JOIN queries to retrieve appointment details, patient history, and doctor-wise patient counts.

Task 2 – Real-Time Application: Online Shopping Database

Scenario:Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

• Retrieve all orders by a user.

• Find the most purchased product.

• Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query

optimization.

Prompt:

Generate SQL code for an Online Shopping Database.

Create tables: Users, Products, Orders, and OrderDetails with proper constraints.

Insert sample data into all tables.

Write JOIN queries to retrieve user orders, most purchased product, and monthly revenue.

Ensure the schema is normalized and include query optimization techniques.

Code:

```
DROP TABLE IF EXISTS OrderDetails;

DROP TABLE IF EXISTS Orders;

DROP TABLE IF EXISTS Products;

DROP TABLE IF EXISTS Users;

CREATE TABLE Users (

    user_id INTEGER PRIMARY KEY,

    name TEXT NOT NULL,

    email TEXT UNIQUE NOT NULL   );

CREATE TABLE Products (

    product_id INTEGER PRIMARY KEY,

    product_name TEXT NOT NULL,

    price REAL CHECK(price > 0));

CREATE TABLE Orders (

    order_id INTEGER PRIMARY KEY,
```

```
    user_id INTEGER,
    order_date DATE NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id));
CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER CHECK(quantity > 0),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id));
INSERT INTO Users VALUES (1, 'Ravi', 'ravi@gmail.com');
INSERT INTO Users VALUES (2, 'Anita', 'anita@gmail.com');
INSERT INTO Products VALUES (1, 'Laptop', 50000);
INSERT INTO Products VALUES (2, 'Mobile', 20000);
INSERT INTO Products VALUES (3, 'Headphones', 2000);
INSERT INTO Orders VALUES (1, 1, '2024-03-10');
INSERT INTO Orders VALUES (2, 1, '2024-03-15');
INSERT INTO Orders VALUES (3, 2, '2024-03-20');
INSERT INTO OrderDetails VALUES (1, 1, 1, 1);
INSERT INTO OrderDetails VALUES (2, 1, 3, 2);
INSERT INTO OrderDetails VALUES (3, 2, 2, 1);
INSERT INTO OrderDetails VALUES (4, 3, 3, 3);
SELECT
    u.name AS user_name,
    o.order_id,
    o.order_date,
    p.product_name,
    od.quantity
FROM Users u
JOIN Orders o ON u.user_id = o.user_id
```

```
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE u.user_id = 1;
SELECT
    p.product_name,
    SUM(od.quantity) AS total_sold
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_name
ORDER BY total_sold DESC
LIMIT 1;
SELECT
    strftime('%Y-%m', o.order_date) AS month,
    SUM(p.price * od.quantity) AS total_revenue
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE strftime('%Y-%m', o.order_date) = '2024-03'
GROUP BY month;
CREATE INDEX idx_user_id ON Orders(user_id);
CREATE INDEX idx_order_id ON OrderDetails(order_id);
CREATE INDEX idx_product_id ON OrderDetails(product_id);
```

Output:

SQL ▼ 1 / 1 1 - 3 of 3

user_name	order_id	order_date	product_name	quantity
Ravi	1	2024-03-10	Laptop	1
Ravi	1	2024-03-10	Headphones	2
Ravi	2	2024-03-15	Mobile	1

SQL ▼ 1 / 1 1 - 1

product_name	total_sold
Headphones	5

SQL ▼ 1 / 1 1 - 1 of 1

month	total_revenue
2024-03	80000.0

Explanation:

This prompt generates a structured e-commerce database with related tables and constraints.It also includes analytical SQL queries using joins for data retrieval and performance optimization.

Task 3 – Library Database

Task:Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

• Retrieve all books currently issued.

• Find overdue books (loan date > 30 days).

• Count number of books loaned by each member.

Prompt:

Generate SQL code for a Library Management System.Create tables: Books, Members, and Loans with proper constraints.Insert sample data into all tables.Write JOIN queries to retrieve issued books, overdue books, and member-wise loan counts.Include indexing strategies for query optimization.

Code:

```
DROP TABLE IF EXISTS Loans;

DROP TABLE IF EXISTS Members;

DROP TABLE IF EXISTS Books;

CREATE TABLE Books (

    book_id INTEGER PRIMARY KEY,

    title TEXT NOT NULL,

    author TEXT NOT NULL  );

CREATE TABLE Members (

    member_id INTEGER PRIMARY KEY,

    name TEXT NOT NULL  );

CREATE TABLE Loans (
```

```
    loan_id INTEGER PRIMARY KEY,
    book_id INTEGER,
    member_id INTEGER,
    loan_date DATE NOT NULL,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id));
INSERT INTO Books VALUES (1, 'DBMS', 'Korth');
INSERT INTO Books VALUES (2, 'Python', 'Guido');
INSERT INTO Books VALUES (3, 'AI Basics', 'John');
INSERT INTO Members VALUES (1, 'Ravi');
INSERT INTO Members VALUES (2, 'Anita');
INSERT INTO Loans VALUES (1, 1, 1, '2024-03-01', NULL);
INSERT INTO Loans VALUES (2, 2, 1, '2024-02-01', '2024-02-20');
INSERT INTO Loans VALUES (3, 3, 2, '2024-02-15', NULL);
-- 1. Retrieve all books currently issued (not returned)
SELECT
    b.title,
    m.name AS member_name,
    l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
SELECT
    b.title,
    m.name AS member_name,
    l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
```

```
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL
AND julianday('now') - julianday(l.loan_date) > 30;
SELECT
    m.name AS member_name,
    COUNT(l.loan_id) AS total_books
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.name;
CREATE INDEX idx_book_id ON Loans(book_id);
CREATE INDEX idx_member_id ON Loans(member_id);
CREATE INDEX idx_loan_date ON Loans(loan_date);
```

Output:

SQL ▼ 1 / 1 1 - 2 of 2

title	member_name	loan_date
DBMS	Ravi	2024-03-01
AI Basics	Anita	2024-02-15

SQL ▼ 1 / 1 1 - 2 of 2

title	member_name	loan_date
DBMS	Ravi	2024-03-01
AI Basics	Anita	2024-02-15

SQL ▼ 1 / 1 1 - 2 o

member_name	total_books
Anita	1
Ravi	2

Explanation:

This prompt creates a structured library database with relationships between books, members, and loans.It also generates JOIN queries and indexing techniques to efficiently retrieve and analyze data.

Task 4: Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

```
-- Original query
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM
Authors WHERE country='UK');
-- Optimized query
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Prompt:

Analyze and optimize SQL queries for better performance.Compare an original query using subqueries with an optimized query using JOIN.Create sample tables and insert data. Ensure both queries return the same result. Apply indexing techniques to improve efficiency.

Code:

```
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Authors;
CREATE TABLE Authors (
    author_id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT   );
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY,
    title TEXT,
    author_id INTEGER,
    FOREIGN KEY (author_id) REFERENCES Authors(author_id));
INSERT INTO Authors VALUES (1, 'John', 'UK');
INSERT INTO Authors VALUES (2, 'Alice', 'USA');
INSERT INTO Books VALUES (1, 'SQL Basics', 1);
INSERT INTO Books VALUES (2, 'Advanced SQL', 1);
```

```
INSERT INTO Books VALUES (3, 'Python Guide', 2);
SELECT *
FROM Books
WHERE author_id IN (
    SELECT author_id
    FROM Authors
    WHERE country = 'UK'    );
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
CREATE INDEX idx_author_id ON Books(author_id);
CREATE INDEX idx_country ON Authors(country);
```

Output:

SQL ▾ 1 / 1 1 - 2 of 2

book_id	title	author_id
1	SQL Basics	1
2	Advanced SQL	1

SQL ▾ 1 / 1 1 - 2 of 2

book_id	title	author_id
1	SQL Basics	1
2	Advanced SQL	1

Explanation:

The optimized query replaces a subquery with a JOIN to improve performance and readability.Indexes further speed up execution, especially for large datasets.

Task 5: University Course Registration

Scenario:SR University needs a course registration system for students enrolling in different courses under faculty supervision.

• Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.

• Define relationships:

• Each student can register for multiple courses.

• Each course is taught by a faculty member.

• Write SQL queries to:

• List all students enrolled in a specific course.

• Find faculty members teaching more than 2 courses.

• Retrieve courses with the highest number of registrations.

Prompt:

Generate SQL code for a University Course Registration System. Create tables: Students, Courses, Faculty, and Registrations with proper relationships. Insert sample data into all tables. Write JOIN queries to list enrolled students, find faculty teaching multiple courses, and identify popular courses. Ensure the schema is normalized and relational.

Code:

```
DROP TABLE IF EXISTS Registrations;

DROP TABLE IF EXISTS Students;

DROP TABLE IF EXISTS Courses;

DROP TABLE IF EXISTS Faculty;

CREATE TABLE Students (

    student_id INTEGER PRIMARY KEY,

    name TEXT NOT NULL  );

CREATE TABLE Faculty (

    faculty_id INTEGER PRIMARY KEY,

    name TEXT NOT NULL   );

CREATE TABLE Courses (

    course_id INTEGER PRIMARY KEY,

    course_name TEXT NOT NULL,

    faculty_id INTEGER,

    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id));

CREATE TABLE Registrations (

    registration_id INTEGER PRIMARY KEY,

    student_id INTEGER,

    course_id INTEGER,

    FOREIGN KEY (student_id) REFERENCES Students(student_id),
```

```
    FOREIGN KEY (course_id) REFERENCES Courses(course_id));
INSERT INTO Students VALUES (1, 'Ravi');
INSERT INTO Students VALUES (2, 'Anita');
INSERT INTO Students VALUES (3, 'Kiran');
INSERT INTO Faculty VALUES (1, 'Dr. Rao');
INSERT INTO Faculty VALUES (2, 'Dr. Sharma');
INSERT INTO Courses VALUES (1, 'DBMS', 1);
INSERT INTO Courses VALUES (2, 'AI', 1);
INSERT INTO Courses VALUES (3, 'Python', 1);
INSERT INTO Courses VALUES (4, 'Networks', 2);
INSERT INTO Registrations VALUES (1, 1, 1);
INSERT INTO Registrations VALUES (2, 2, 1);
INSERT INTO Registrations VALUES (3, 3, 2);
INSERT INTO Registrations VALUES (4, 1, 2);
INSERT INTO Registrations VALUES (5, 2, 3);
INSERT INTO Registrations VALUES (6, 3, 4);
SELECT
    s.name AS student_name,
    c.course_name
FROM Registrations r
JOIN Students s ON r.student_id = s.student_id
JOIN Courses c ON r.course_id = c.course_id
WHERE c.course_id = 1;
SELECT
    f.name AS faculty_name,
    COUNT(c.course_id) AS total_courses
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.name
HAVING COUNT(c.course_id) > 2;
```

```
SELECT
    c.course_name,
    COUNT(r.student_id) AS total_students
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY total_students DESC
LIMIT 1;
```

Output:

SQL ▼ 1 / 1 1 - 2 o

student_name	course_name
Ravi	DBMS
Anita	DBMS

SQL ▼ 1 / 1 1 - 1 of

faculty_name	total_courses
Dr. Rao	3

SQL ▼ 1 / 1 1 - 1 of

course_name	total_students
DBMS	2

Explanation:

This prompt creates a relational database linking students, courses, and faculty through registrations.It uses JOIN queries and aggregation to analyze enrollments and teaching workload.